画像処理実験 第 4 回

学生番号: 09B24490
加藤弘都 (KATO, HIROTO)

2026 年 10 月 22 日

1　課題 4

合成画像を作るために，ある画像から特徴点を取り，特徴点を起点に画像を合成したい．まず各画素における勾配を調べる．入力画像 $I(x,y)$ を x 偏微分，y 偏微分するが今回は以下の差分で近似する．

$$\begin{pmatrix} I_x(x,y) \\ I_y(x,y) \end{pmatrix} = \begin{pmatrix} I(x+1,y) - I(x-1,y) \\ I(x,y+1) - I(x,y-1) \end{pmatrix}$$

つぎに，勾配 $I(Ix, Iy)$ の分布を楕円で近似するとき，楕円の形状は以下のような点群の共分散行列で表現できる．

$$\begin{pmatrix} I_{xx} & I_{xy} \\ I_{xy} & I_{yy} \end{pmatrix} = \frac{1}{|\Omega(x,y)|} \sum_{\Omega(x,y)} \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}$$

この行列の固有値は楕円の長半径と短半径を表す．このとき，固有値（長半径と短半径）は勾配の分布をまとめていることから，画像が変化した方向，変化した大きさを表している．

本章では小さい方の固有値（短半径）を `TKfilter` 関数で返している．画像はエッジ（上下で色が変わる部分など）の場合，長半径は大きいが短半径が小さい（縦のみが変化），しかし，角（エッジが二方向から交わる）の場合，長半径と短半径の両方が大きい（縦にも横にも変化）．このことから，長半径だけではエッジか角かの区別がつかないが，短半径なら区別がつくため，特徴点を発見するのに向いているのである．

1.1　[4-2]

1.1.1　自分の結果と「特徴らしさ画像」を照合

$A = \begin{bmatrix} Ixx & Ixy \\ Ixy & Iyy \end{bmatrix}$

勾配 (`Ix,Iy`) を用いて上記のような共分散行列をつくる．この行列の固有値は勾配の分布を近似した楕円の長半径と短半径を表す．このことより，固有値の二次方程式を解の公式で解くと以下のようになる．ただし，以下は最小の固有値を表す．

$\lambda_{\min} = \frac{I_{xx}+I_{yy}-\sqrt{(I_{xx}-I_{yy})^2+4I_{xy}^2}}{2}$

コードでは以下のように表現している．

```
1:      dstf[y,x]=(ixx+iyy-np.sqrt((ixx-iyy)*(ixx-iyy)+4*ixy*ixy))/2
```

この `dstf` を画像として表示させると図**??**のようになる．

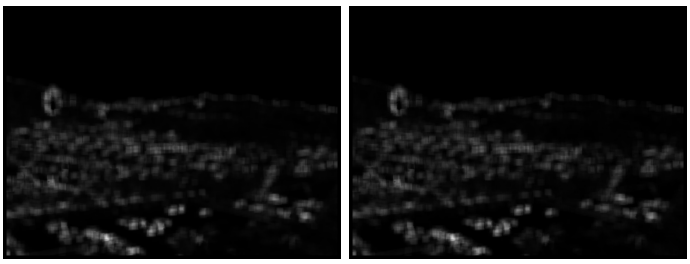


図 1　「特徴らしさ画像」（左）と自分の結果（右）

図**??**の二枚の画像を比較すると，私の結果は「特徴らしさ画像」と同じような特徴を捉えていることがわかる．

1.1.2　計算過程の確認

(1) では，I に x，y についてそれぞれの偏微分をしている．偏微分には以下のように差分で近似して表現している．

```
1:      Ix[y,x]=src[y,x+1  ,1] - src[y,x-1  ,1]
2:      Iy[y,x]=src[y+1,x  ,1] - src[y-1,x  ,1]
```

(2) では (3) で共分散行列を求めるのに使用する，Ix*Ix，Ix*Iy，Iy*Iy を求めている．

(3) では，(4.1) の右辺を表している．blur 関数で Ixx，Ixy，Iyy に平滑化を行っている．これにより，共分散行列を表す要素 Ixx，Iyy，Ixy を求めることができた．

1.1.3　2 重ループを除去

conv2d 関数を利用する．この関数で使用している convolve2d 関数は引数に画像 im，行列 k をうけとる．そして，画像の畳込み演算を行う関数である．k が以下のとき，x について偏微分を行う．y について偏微分する場合，以下の行列を転置する．

$k = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$

また，平滑化する場合は大きさ（今回は 3x3 行列）に応じて以下のような行列にすることで実現できる．

$$k = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

このことを踏まえて，(1) の偏微分を以下のように変更できる．この変更により，二重ループを除去できる．

```
1:  Ix=conv2d(src[...,1], np.array([[1,0,-1]],'f'))
2:  Iy=conv2d(src[...,1], np.array([[1],[0],[-1]],'f'))
```

また，blur 関数も同様に以下のように変更し，二重ループを除去できる．

```
1:  X=W+1+W
2:   return conv2d(im, np.full((X,X),1/(X*X),dtype='f') )
```

また，(2) と (4) で行列同士の掛け算をするときに for 文の二重ループを利用している．行列同士の演算は A*B のように単純に表せるため，この二重ループも変更する．

1.1.4 偏微分・平滑化の実現

$$A(x,y) = \begin{bmatrix} a11 & a12 & a13 \\ a21 & a22 & a23 \\ a31 & a32 & a33 \end{bmatrix}$$

上記のように $A(x,y)$ を定義する．

まず，偏微分について調べる．`A(x,y)*[[1,0,-1]]` を式 (4.2) に従って次のように展開する．

$$A(x,y) * [1,0,-1] = \begin{bmatrix} a13 - a11 \\ a23 - a21 \\ a33 - a31 \end{bmatrix} = I(x+1,y) - I(x-1,y)$$

これは偏微分を差分で近似した式と一致するから，`x` について偏微分していると言える．また，`y` について偏微分する場合，`B` を `[[1],[0],[-1]]` のようにすれば計算できる．

次に，平滑化について考える．画像の平均化について考えたいので，畳み込む 2 つ目の行列 `B` を以下のように定義する．

$$B = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

`A(x,y)*B` を式 (4.2) に従って次のように展開する．

$$A(x,y)*B = \begin{bmatrix} \frac{a_{11}+a_{12}+a_{21}+a_{22}}{9} & \frac{a_{11}+a_{12}+a_{13}+a_{21}+a_{22}+a_{23}}{9} & \frac{a_{12}+a_{13}+a_{22}+a_{23}}{9} \\ \frac{a_{11}+a_{12}+a_{21}+a_{22}+a_{31}+a_{32}}{9} & \frac{a_{11}+a_{12}+a_{13}+a_{21}+a_{22}+a_{23}+a_{31}+a_{32}+a_{33}}{9} & \frac{a_{12}+a_{13}+a_{22}+a_{23}+a_{32}+a_{33}}{9} \\ \frac{a_{21}+a_{22}+a_{31}+a_{32}}{9} & \frac{a_{21}+a_{22}+a_{23}+a_{31}+a_{32}+a_{33}}{9} & \frac{a_{22}+a_{23}+a_{32}+a_{33}}{9} \end{bmatrix}$$

このとき，計算結果の行列の中央の要素について考える．

$$\frac{a_{11}+a_{12}+a_{13}+a_{21}+a_{22}+a_{23}+a_{31}+a_{32}+a_{33}}{9}$$

この値は `A` のすべての値の和を 9 で割っている．このため，画像の畳込みで中央の 1 画素だけを平滑化できることがわかる．

1.1.5 計算時間・コード量・拡張性等

??節で作成したコードと以降に変更したコードの 2 つについて比較する．

まず，計算時間についてまとめる．表**??**では変更前後のコードで (1) から (4) までの実行時間を記録している．表を見ると，(3) 以外のすべて計算時間が早くなっている．

表 1　変更前後の時間の変化

	(1)	(2)	(3)	(4)
変更前	0.638	0.700	0.240	1.209
変更後	0.062	0.009	0.613	0.003

(3) は `blur` 関数のみ変更している．画像の畳込みを利用しているため，各画素ごとに `k` を画素に掛けて足している．変更前では配列演算のみを行っているが，変更後は畳み込み演算を中心に実行するため単純な平均化に特化した処理ではなく，比較すると遅くなってしまったと考える．

つぎにコード量について，より簡潔になったと考える．二重ループをなくすために，`conv2d` 関数を使用したこと，行列計算を for 文なしで計算したことなどにより，簡潔に表現できた．

最後に拡張性について，高くなったと考える．`conv2d` 関数を定義したことで，他に畳み込みを利用した

処理が必要になったときに関数を呼び出すだけで処理を実行できるようになる.

1.1.6 blur 関数のベクトル化

```
1: def blur(im,W=7): # a.k.a. Box blur
2:  out=np.zeros_like(im)
3:
4:  for v in range(-W,W+1):
5:   for u in range(-W,W+1):
6:    out[W:-W,W:-W]+=im[v+W:im.shape[0]+v-W,u+W:u+im.shape[1]-W]
7:  return out/((2*W+1)*(2*W+1))
```

二段階化したコードは以下である.

```
1:  out=np.zeros_like(im)
2:  t=np.zeros_like(im)
3:
4:  for v in range(-W,W+1):
5:    t[W:-W,:]+=im[v+W:v+im.shape[0]-W,:]
6:  t=t/(2*W+1)
7:  for u in range(-W,W+1):
8:   out[:,W:-W]+=t[:,u+W:u+t.shape[1]-W]
9:  return out/(2*W+1)
```

2 感想

参考文献